

Projektuppgift

Programmering i C#.NET

Projekt
ExpenseTracker

Ramona Reinholdz



Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. Campus Sundsvall Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Ramona Reinholdz, rare2400@student.miun.se

Utbildningsprogram: Webbutveckling, 120 hp

Huvudområde: Datateknik

Termin, år: HT, 2025

Sammanfattning

Detta projekt går ut på att skapa en konsolbaserad applikation med programmeringsspråket C# och ramverket .NET. Applikationen är utvecklad för att hjälpa användare att hantera och få en överblick över sina utgifter. Genom programmet kan användaren lägga till, visa, ändra och ta bort befintliga utgifter, samt se utgifternas beräknade totalsumma. Utgifterna lagras i en JSON-fil, vilket gör att informationen finns kvar även efter att programmet avslutas. För att detta används .NET:s inbyggda metoder för serialisering och deserialisering av data till och från JSON-format.

Applikationen innehåller ett konsolbaserat användargränssnitt som är utformat för att vara tydligt och enkelt att förstå. Användarens inmatningar kontrolleras och valideras innan de hanteras, vilket minskar risken för felaktig data och bidrar till en stabil och pålitlig applikation.

Innehållsförteckning

1	Inledning.....	4
2	Teori	5
2.1	C#	5
2.2	.NET.....	5
2.3	Objektorienterad programmering med C#	6
2.4	CRUD.....	6
3	Metod.....	7
3.1	Verktyg	7
3.2	Utveckling	7
4	Konstruktion	8
4.1	Planering	8
4.2	Struktur och uppbyggnad	9
4.3	Användagränssnitt.....	9
5	Resultat	11
6	Slutsatser.....	12
7	Källförteckning	13
8	Bilagor	14
8.1	Bilaga A. Flödesschema för ExpenseTracker	14

1 Inledning

Många har behov att få bättre koll på sin utgifter och se hur mycket pengar man spenderar över tid. Det kan vara allt från vardagliga kostnader som mat, transport eller andra små utgifter som annars lätt tappas bort i mängden. Syftet med projektet är att utveckla en konsolbaserad applikation, där användaren kan registrera och hantera sina utgifter på ett enkelt sätt. Applikationen ska göra det möjligt för användaren att lägga till, visa, ändra och ta bort utgifter, samt att se en beräknad totalsumma över alla sparade utgifter. Varje utgift ska struktureras upp i belopp, kategori, beskrivning och datumet för utgiften, med datatyper passande till egenskapen.

Konsolapplikationen ska utvecklas med programmeringsspråket C# med plattformen .NET. Data ska lagras i en JSON-fil, vilket gör att informationen finns kvar även efter programmet avslutas. För detta används .NET:s inbyggda funktioner för att serialisera och deserialisera data till och från JSON-format. Ett konsolbaserat användargränssnitt ska skapas för att göra programmet lätt att förstå och använda. Användarens inmatningar ska kontrolleras och valideras innan dem behandlas av applikationen. Detta minskar risken för felaktig data och bidrar till att applikationen fungerar stabilt.

2 Teori

2.1 C#

Programmeringsspråket C# är utvecklat av Microsoft och används tillsammans med ramverket .NET. Det kan användas för att skapa applikationer till många olika enheter så som telefoner, servrar och datorer. C# är inspirerat av programmeringsspråken C++ och Java och fokuserar på objektorienterad programmering. För att förbättra upplevelsen, så utvecklas C# konstant och nya versioner kommer löpande. När man har kodat C# exekveras den med hjälp av CLI (Common Language Infrastructure) som kompilerar koden till ett mellanspråk. Detta gör sedan att plattformar med CLR (Common Language Runtime) kan exekvera koden [1]. C# är ett strikt typat språk där variabler som deklarerats kan, som figur 1 visar, ha en angiven datatyp. Detta leder till att man direkt kan se och åtgärda fel innan programmet körs [2]. Det finns ett alternativ där man låter kompilatorn bestämma datatypen. Då används `var` för att deklarera variabeln istället [1].

Figur 1. Variabler deklarerade med datatyper i C#.

```
string name = "Ella";  
int numb = 26;  
bool isTrue = true;
```

I C# finns funktionaliteten LINQ (Language Integrated Query) som fungerar tillsammans med allt från databaser och collections till JSON. Det finns till exempel metoder för att sortera (`OrderBy/OrderByDescending`) och filtrera data (`Where`). LINQ inkluderas även av helpers som exempelvis som figur 2 visar kan räkna ut ett totalvärde eller antalet (`Count`) element i en lista eller tabell [1].

Figur 2. Helpern Sum räknar ut ett totalvärde av ett antal tal.

```
var numbers = new[] { 1, 5, 3, 77, 46 };  
int total = numbers.Sum();  
Console.WriteLine($"Total: {total}");
```

```
ramona@macbook dotnet run  
Total: 132
```

2.2 .NET

Ramverket .NET är en plattform för utvecklare med öppen källkod som kan köras med flera programmeringsspråk men används främst tillsammans med C#. Med plattformen kan man exempelvis utveckla konsolapplikationer, webbapplikationer och desktopprogram [3]. Det är .NET SDK som består av verktyg för exekvering av kod, där `dotnet run` och `dotnet build` kör och kompilerar koden så CLR kan köra den [1].

Tillsammans med att C# utvecklas med nya versioner kommer även uppdaterade versioner av ramverket .NET [1]. Underhållet av plattformen görs i ett samarbete mellan ett globalt community och Microsoft. Plattformen bygger på tre grundpelare: runtime, bibliotek och språk. Kodens runtime i .NET är högpresterande och det används i storskaliga applikationer i produktion. Då hanteras bitar som minne, säkerhet och exekvering av programkod. Ett bibliotek

av omfattande färdiga funktioner tillhandhålls också av .NET. Det kan vara funktioner som filhanterng, datum och tid eller hantering av JSON [3]. Till exempel används `System.Text.Json` i detta projekt för JSON-serialisering [4]. Som tidigare nämnt är C# det primära programmeringsspråket, men .NET stöder både statisk och dynamisk kod. Plattformen är utformad för att leverera produktivitet, prestanda, säkerhet och tillförlitlighet, Detta stöds av de regelbundna uppdateringarna som ger användarna säkrare mer tillförlitlig applikationer skapade med .NET [3].

2.3 Objektorienterad programmering med C#

C# är ett objektorienterat programmeringsspråk därmed är .NET-plattformen anpassad för utveckling med objektorienterad programmering (OOP). Det innefattar begrepp som klasser, objekt och instanser där en klass kan användas som en instans för att skapa objekt. Alltså kan man skapa flera likadana objekt med en klass. När man skapar en klassdefinition ska den innehålla fält, properties och metoder/funktioner. Metoder eller funktioner ger en returdatatyp i det objekt som de skapar.

Datafält och metoder definieras med en åtkomstkontroll i form av `public`, `protected`, `private` eller `internal`. I detta projekt används `public` och `private`, där den förstnämnda ger åtkomst för alla medan `private` endast kan nås av det egna objektet. Inom objektorienterad programmering används properties som setters och getters för att begränsa åtkomst till fält. Dessa kan man som i figur 3 placera i en egen klass, där kan data hämtas genom eller lagras. Vill man skriva ut ett inlägg med ägaren genom figur 3 så kan man, efter man hämtat klassen, skriva `Console.WriteLine($"{post.Owner} - {post.PostText}");` [5].

Figur 3. Klassen `post` med setters och getters på ägare och inlägg.

```
public class Post
{
    0 references
    public string? Owner { get; set; } //ägaren
    0 references
    public string? PostText { get; set; } //inlägget
}
```

2.4 CRUD

CRUD står för Create, Read, Update och Delete och är grundläggande åtgärder som genomförs i ett projekt som hanterar datalagring. De fyra CRUD-operationerna innebär att man ska kunna lägga till ny data samt läsa ut, uppdatera och radera befintlig data. Att använda CRUD vid utveckling är en grundsten för att skapa stabila och effektiva applikationer [6].

3 Metod

3.1 Verktyg

Verktygen som används vid arbetet med applikationen är Visual Studio Code som utvecklingsmiljö samt git och Github för versionshantering. Under planeringsstadiet skapas ett flödesschema och en UML över applikationens klassstruktur med hjälp av draw.io i Visual Studio Code. Konsolapplikationen utvecklas med programmeringsspråket C# med plattformen .NET 9.0.304, som används för att bygga och kör applikationen [3].

3.2 Utveckling

Utvecklingen genomförs med objektorienterad programmering där koden struktureras i klasser med tydliga ansvarsområden. Detta underlättar även läsning och felsökning av koden. Data representeras i en modellklass, medan logiken och filhanteringen hanteras i varsin service-klass. Program.cs ansvarar sedan för användargränssnitt och användarens interaktioner.

Färdiga funktioner som .NET erhåller används även för att effektivisera utvecklingen av applikationen. Till exempel används biblioteket `System.Text.Json` för att serialisera och deserialisera JSON-data [4]. Applikationen delas upp för att hantera olika moment enligt CRUD [6] där data kan läggas till, presenteras, uppdateras och tas bort. Även inbyggd funktionalitet i C# såsom LINQ:s metoder och helpers används, för att skriva ut data eller beräkna totalsummor.

Under utvecklingen testas applikationen regelbundet för att upptäcka och åtgärda eventuella fel. Indata från användaren kontrolleras med hjälp av if- och else-satser, för att säkerställa att giltiga värden matats in [5].

4 Konstruktion

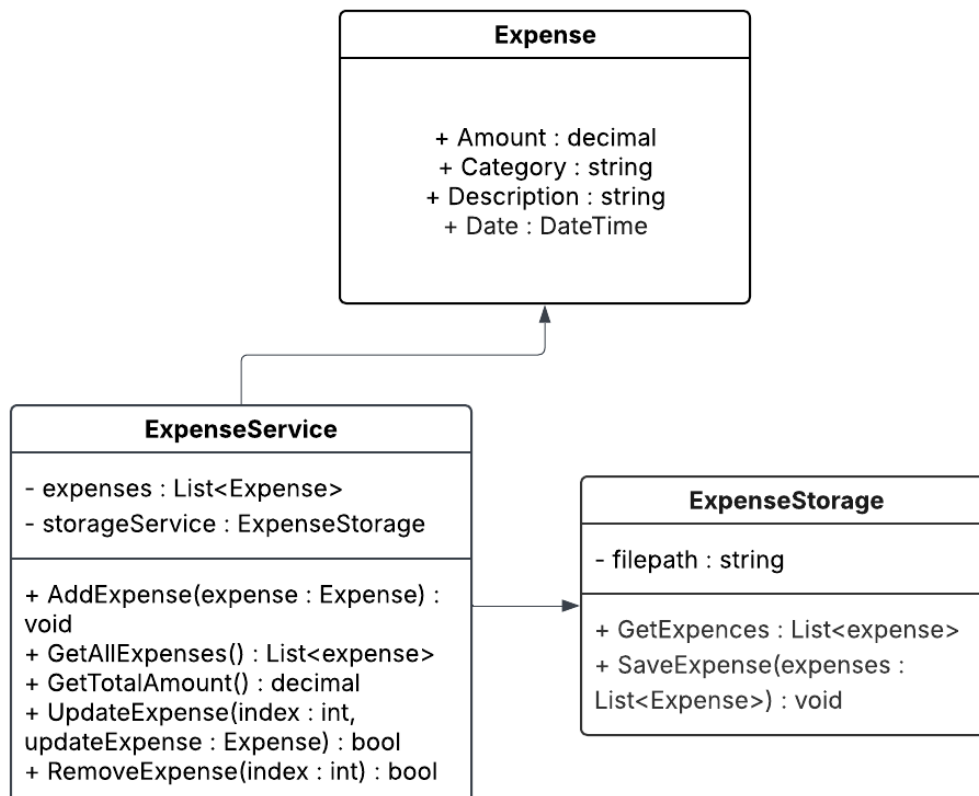
4.1 Planering

Den första delen av arbetet med projektet bestod av planering av applikationens funktionalitet och struktur. Målet var att utveckla en konsolbaserad applikation som gör det möjligt för användare att hantera och få en överblick över sina utgifter på ett enkelt och tydligt sätt. För att skapa en tydlig bild av programmets flöde och användarinteraktioner togs ett flödesschema fram, vilket återfinns i Bilaga A. Flödesschemat visar hur applikationen startar, hur menyval presenteras och hur användarens val leder vidare till olika funktioner.

Applikationens flöde är uppbyggt efter CRUD-principen [6], där användaren kan lägga till, visa, uppdatera och ta bort utgifter. Ett femte menyval används för att avsluta programmet. Om användare matar in ett ogiltigt menyval, visas ett felmeddelande och användaren skickas tillbaka till huvudmenyn. Flödesschemat användes under utvecklingen för att hålla ett konsekvent, logiskt användarflöde.

För att få en tydlig överblick över applikationens struktur och klassernas ansvar skapades ett UML-diagram över projektets klasser och deras relationer. Som figur 4 visar består detta projekt av 3 klasser som hanterar data, logik och filhantering.

Figur 4. UML-diagram över projektets klasser



4.2 Struktur och uppbyggnad

Utvecklingen av applikationen inleddes genom att skapa ett nytt konsolprojekt med hjälp av .NET CLI med kommandot `dotnet new console`. Applikationen körs därefter med `dotnet run`. Under utvecklingsprocessen användes versionshantering med Git tillsammans med Github. En `.gitignore`-fil skapades med kommandot `dotnet new gitignore`, för att exkludera filer som inte ska versionshanteras [7].

Projektet är utvecklat enligt objektorienterad programmering och är uppbyggd på tre klasser med tydliga ansvarsområden. Koden delas upp i mapparna Models och Services, för att skapa en tydlig struktur som gör det enklare att underhålla och hitta i projektet. I mappen Models finns klassen Expense, som representerar en utgift. Klassen innehåller egenskaper för belopp, kategori, beskrivning och datum, där datum default-värde som sätts till aktuellt datum om inget annat anges. Egenskaperna är implementerade med setters och getters. Det gör det möjligt att hämta och uppdatera värden på ett kontrollerat sätt, samtidigt som det bidrar till att kapsla in datan på ett strukturerat sätt.

Mappen Services innehåller två klasser. Klassen ExpenseTracker ansvarar för logiken kring hantering av utgifter enligt CRUD. Den hanterar funktioner för att lägga till, hämta, ta bort och ta bort utgifter samt räkna ut den totala summan av alla utgifter. När en ny uppgift skapas i applikationen instansieras ett objekt från Expense-klassen, som skickas vidare till service-klassen. I denna klass används LINQ för att beräkna totalsummor, samt för att utföra kontroller på listan med sparade utgifter.

Den andra service-klassen, ExpenseStorage, ansvarar för filhanteringen. Klassen innehåller metoder för att läsa in och spara utgifter till en JSON-fil. Metoden `GetExpenses` kontrollerar att filen existerar och innehåller data. Om filen saknas eller är tom returneras en tom lista. Finns data i filen läses innehållet in och deserialiseras med hjälp av `System.Text.Json`. Metoden `SaveExpenses` tar emot en lista med utgifter, serialiserar den till JSON-format och sparar informationen till filen. Genom att separera filhanteringen från logiken blir applikationen mer flexibel och lättare att vidareutveckla.

4.3 Användargränssnitt

I Program.cs utvecklas det konsolbaserade användargränssnittet, som fungerar som applikationens startpunkt. Programmet använder en while-loop som gör att menyn i figur 5 visas tills det att användaren väljer att avsluta programmet. Inuti while-loopen används en switch-sats som hanterar användarens menyval enligt flödesschemat i bilaga A och anropar rätt funktion baserat på det valda alternativet.

Figur 5. Användargränssnittets huvudmeny.

```
Välkommen till ExpenseTracker!  
  
Här kan du se och hantera dina utgifter,  
eller varför inte skapa din egen budget?  
  
=====
```

1. Lägg till utgift
2. Visa dina utgifter
3. Ändra utgift
4. Ta bort utgift

X. Avsluta

Välj ett alternativ:
█

Användarens indata valideras innan den bearbetas vidare i applikationen. Belopp kontrolleras för att säkerställa att giltiga och positiva numeriska värden matas in. Strängvärden som kategori och beskrivning kontrolleras, så att de inte lämnas tomma. Datum kontrolleras med hjälp av `DateTime.TryParse`, där det aktuella datumet används som default om användaren inte anger ett datum. Denna validering minskar risken för felaktig data och leder till en stabil och användarvänlig applikation. När data har validerats skapas ett `Expense`-objekt som skickas vidare till service-klassen `ExpenseStorage` för bearbetning och lagring. Vid visning av utgifter presenteras informationen i form av en tabell i konsolen sorterad efter datum, tillsammans med en beräknad totalsumma för alla utgifter.

5 Resultat

Projektet resulterar i en fungerande konsolbaserad applikationen där användaren kan hantera sina personliga utgifter på ett strukturerat sätt. Applikationen uppfyller projektets syfte genom att erbjuda funktionalitet för att lägga till, isa, ändra och ta bort utgifter, samt beräkna en totalsumma över lagrade utgifter.

När programmet startas laddas lagrade utgifter automatiskt in från en JSON-fil. Om ingen fil finns, eller om filen saknar data, startar applikationen om med en tom lista. Detta gör att applikationen kan användas både när den körs för första gången och vid senare tillfällen utan att data går förlorad. Vill användaren lägga till en utgift får den ange belopp, kategori, beskrivning och datum. Inmatningen kontrolleras innan den sparas, där beloppet måste vara ett giltigt positivt tal, och textfält inte får lämnas tomma. Om datum inte anges används dagens datum per automatik. När en utgift har lagts till visas en bekräftelse för användaren och information sparas direkt till JSON-filen. Väljer användaren att visa alla utgifter presenteras dem i en tabell i konsolen, sorterade efter datum. Under tabellen visas även den totala summan av alla utgifter, vilket ger användaren en tydlig överblick över sin ekonomi.

Applikationen ger även möjlighet till att uppdatera befintliga utgifter. Användaren väljer vilken utgift som ska ändras genom att ange dess nummer i listan som visas i tabellen. Vid uppdatering visas tidigare värden, och användaren kan välja att ändra ett eller flera fält, vill den behålla befintliga värden kan lämna inmatningen tom. Efter uppdatering spara ändringarna till JSON-filen och en bekräftelse visas. Ett menyval ger även möjligt att ta bort en utgift genom att ange vilken som ska raderas med numren som skrivs ut i en tabell. Applikationen hanterar ogiltiga inmatningar genom att visa felmeddelanden och återgå till huvudmenyn utan att krascha eller påverka sparad data.

Sammanfattningsvis resulterar projektet i en stabil användarvänlig konsolbaserad applikation som uppfyller det angivna syftet för arbetet. Applikationen hanterar data på ett säkert sätt, följer CRUD-principen och använder objektorienterad programmering med tydlig uppdelning av logik, data och användargränssnitt.

6 Slutsatser

Arbetet med detta projekt har varit både lärorikt och utvecklande för mig. Projektets syfte var att skapa en konsolapplikation där användaren kan hantera sina utgifter och få en överblick över sina utgifter, vilket jag tycker den färdiga lösningen har uppfyllt. Applikationen gör det möjligt att lägga till, visa, uppdatera och ta bort utgifter, samt se en beräknad totalsumma. Detta ger användaren en tydlig bild av sina utgifter över tid, och skulle även alternativt kunna vara ett sätt att göra applikationen till ett budgetverktyg.

Under utvecklingsprocessen har jag haft stor nytta av kunskaper jag fått från kursen, särskilt inom objektorienterad programmering. Genom att dela upp applikationen i flera klasser med tydliga ansvarsområden har jag fått en bättre förståelse för hur struktur påverkar både läsbarhet och underhåll av koden. Det har även varit intressant att hitta funktioner som kommer med .NET och underlättar mycket i arbetet. Till exempel hur enkelt det är att formatera datum efter önskemål, samt serialisering av data till och från JSON-format. Arbetet med validering av användarens inmatning har också varit en viktig del för applikationen, då det minskar risken för felaktig data och bidrar till en stabilt fungerande applikation.

Om jag hade haft mer tid hade jag velat vidareutveckla applikationen med fler funktioner. Framför allt hade jag velat lägga till bättre möjligheter för sortering och filtrering av utgifter, framför allt per kategori eller datumintervall. I längden kan det bli stora mängder data som ska hanteras och det kan vara svårt att ha allt i en enda tabell, så det hade jag gärna gjort. Jag hade även kunnat tänka mig att utveckla tydligare sammanställningar av data, som exempelvis totalsummor per månad eller kategori.

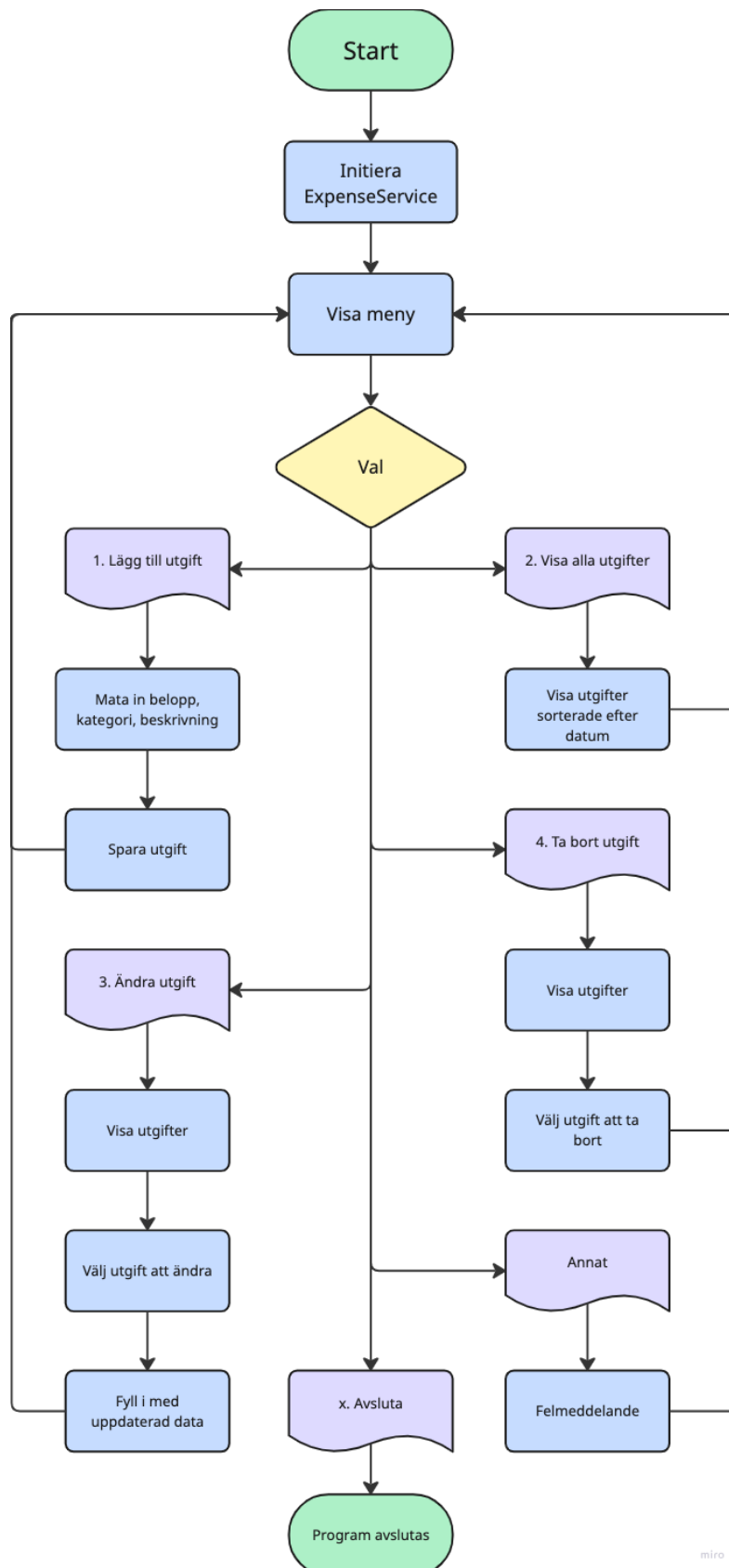
Avslutningsvis är jag nöjd med resultatet och känner att projektet har uppfyllt sitt syfte. Arbetet har gett mig en djupare förståelse för C# och .NET, samt hur objektorienterad programmering kan användas i praktiken. Projektet har varit roligt att arbeta med och har stärkt mitt intresse för att arbeta mer med det i utveckling av framtida applikationer.

7 Källförteckning

- [1] Mittuniversitetet, Hasselmalm M. DT071G – C#. [Video]. Sundsvall: Mittuniversitetet; 9 september 2025 [citerad 2026-01-05]. Hämtad från: https://play.miun.se/media/DT071G%20-%20C/0_0hse214e
- [2] Price M. C# 12 and .NET 9. 9 ed. Birmingham: Packt Publishing Ltd.; 2024.
- [3] Microsoft Learn. Introduction to .NET [Internet]. Microsoft Learn; 2026 [uppdaterad 2024-01-10; citerad 2026-01-07] Hämtad från: <https://learn.microsoft.com/en-us/dotnet/core/introduction>
- [4] Microsoft Learn. JSON serialization and deserialization in .NET - overview [Internet]. Microsoft Learn; 2026 [uppdaterad 2025-01-29; citerad 2026-01-07] Hämtad från: <https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/overview>
- [5] Mittuniversitetet, Hasselmalm M. DT071G C# OOP. [Video]. Sundsvall: Mittuniversitetet; 23 september 2025 [citerad 2026-01-07]. Hämtad från: https://play.miun.se/media/DT071G%20-%20C/0_0hse214e
- [6] Shkutnik M. What is CRUD? A Developer's Guide to the Basics of Web AppOperations [Internet]. UI Bakery; 2026 [uppdaterad 2024-11-15; citerad 2026-01-06] Hämtad från: <https://uibakery.io/blog/what-is-crud>
- [7] Pienkowski R. Easy to create .gitignore for the dotnet developers [Internet]. DEV community; 2026 [uppdaterad 2019-12-01; citerad 2026-01-09] Hämtad från: <https://dev.to/rafalpienkowski/easy-to-create-gitignore-for-the-dotnet-developers-1h42>

8 Bilagor

8.1 Bilaga A. Flödesschema för ExpenseTracker



miro